Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Slides include material from Introduction to Algorithms, 4th Edition, MIT Press by Cormen, Leiserson, Rivest, and Stein.

Lecture 7

- Recommended reading:
  - Chap. 6 (Heaps, Heapsort, Priority Queues)
    Appendix C.4 (just the material on geometric distribution)
    Notes on probability posted on EdStem
- Recommended reading ahead:
  - Chap. 5 (Randomized algorithms, Sections 5.1-5.3)
    Chap. 7 (Quicksort)

Expected number of independent trials to get success?

- Consider repeated independent trials where probability of success in each trial is p
  - e.g., flip fair coin, success is Heads $p = \frac{1}{2}$
  - e.g., roll fair 6-sided die, success is getting 2 $p = \frac{1}{6}$
- Expected number of trials to get first success is $\frac{1}{p}$

- Consider repeated independent (Bernoulli) trials where probability of success in each trial is p
  - e.g., flip fair coin, success is Heads $p = 1/2$
  - e.g., roll fair 6-sided die, success is getting 2 $p = 1/6$
- Expected number of trials to get first success is $1/\mathrm{p}$

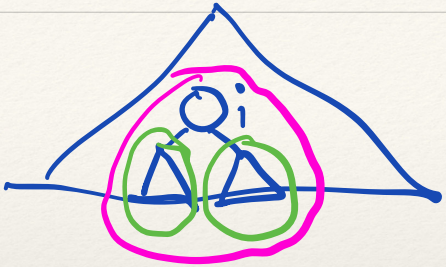
- Recall: $\sum_{i=1}^{\infty} \frac{i}{2^i} = 2$ Expected number of flips of fair coin to get first Heads
- $\sum_{i=1}^{k} \frac{i}{2^i} < 2$ for any integer $k > 0$ ✓

Heaps and Heapsort

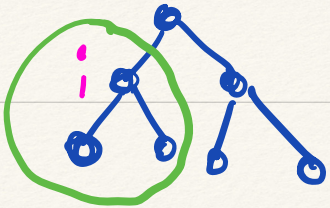
Relevant Heap Operations

- (For Max-Heap)
  - Max-Heap-Heapify(A,i) // Helper function, see next slide
  - BuildMaxHeap(A,n) // take array A of size n and turn it into max-heap
  - Max-Heap-Extract-Max(A) // remove max element from heap

Max-Heapify(A,i)



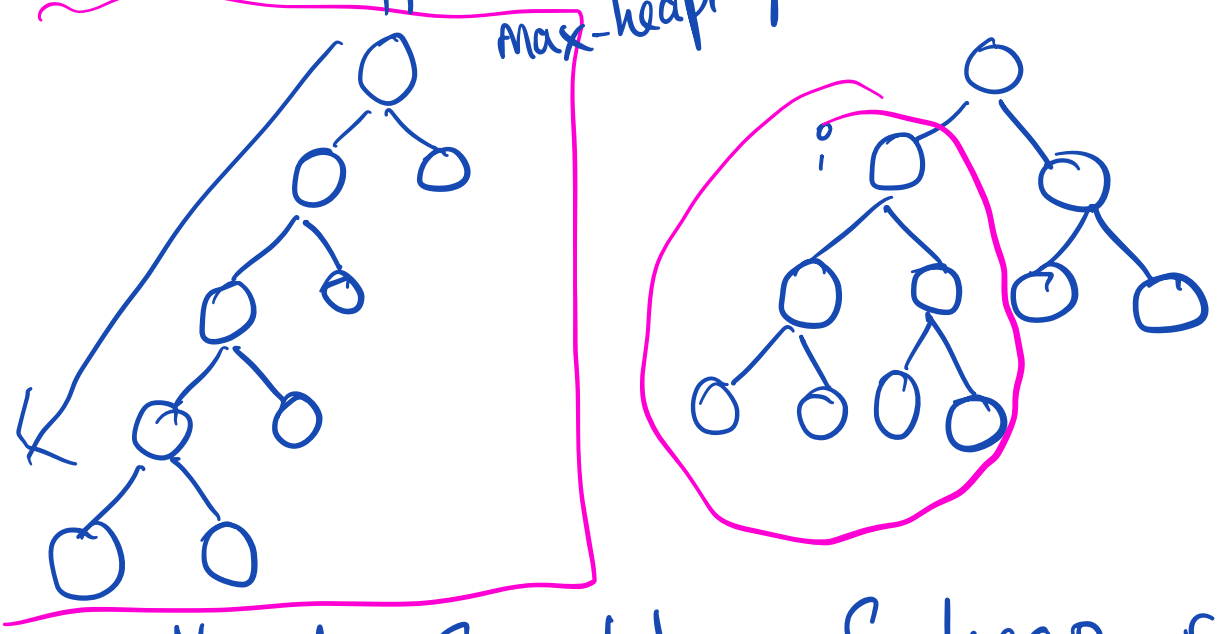
- Assumes left and right children of node i are roots of max-heaps
- But A[i] might be less than A[left(i)] and A[right(i)]
- Max-Heapify(A,i) moves element in node i down in the heap to make node i the root of a max-heap



Worst-case

- Running time of Max-Heapify
  - $\theta(h)$ where h is height of subtree with root at node i [height here is max # edges from node to leaf]
  - $= \theta(\log n)$ if subtree has n elements
- How to convert completely unsorted array into max-heap?
  - Starting from bottom level of corresponding tree
    - Run Max-Heapify on all nodes at that level, so all subtrees rooted at that level are max-heaps.

Never happens for max-heapify



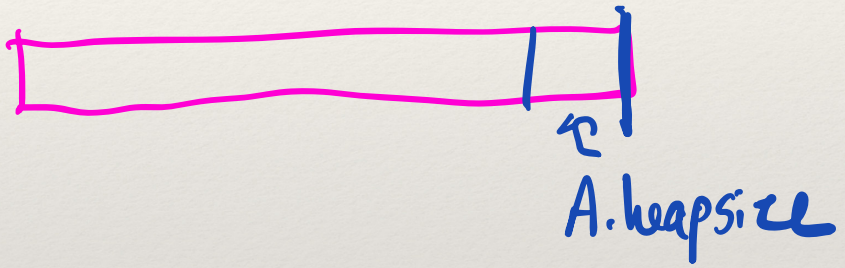
Height of subtree of heap rooted at node i is

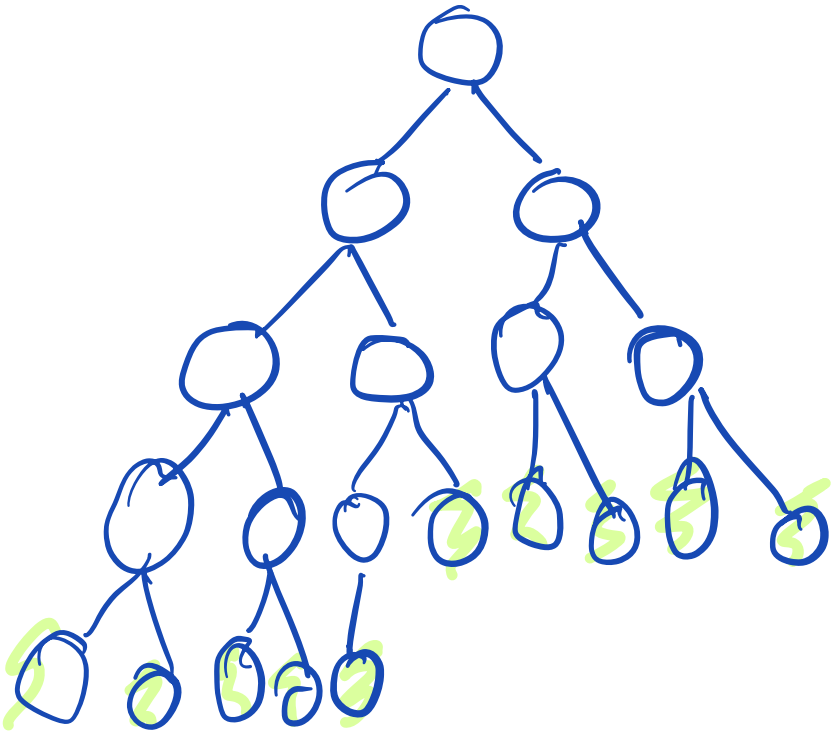
Θ(log #nodes in that subtree)

Pseudocode for Build-Max-Heap

Take unordered array A[1:n] & has size n
+ put it in "heap order"

- Build-Max-Heap(A, n)
  - A.heapsize = n
  - for i = $\lfloor n/2 \rfloor$ downto 1
    - Max-Heapify(A,i)



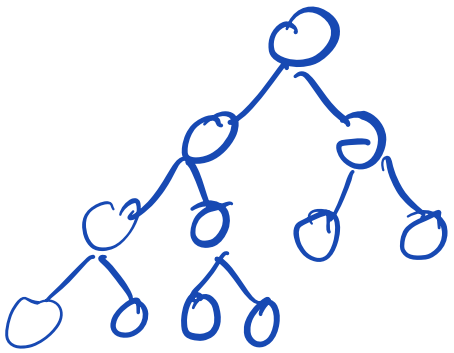


Runtime of Build-Max-Heap

- Height of entire tree is
- Build-Max-Heap runs Max-Heapify n/2 times
- Max-Heapify runs in time $\theta(h)$, where h is height of subtree
- So each Max-Heapify runs in time O(log n)

⇒ Build-Max-Heap runs in time O(n log n)

What is the height of the ~~root~~ tree of ~~the~~ for A

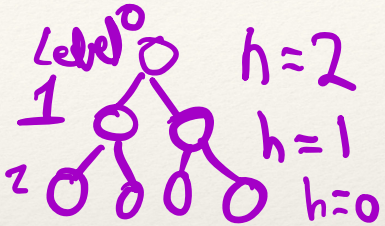


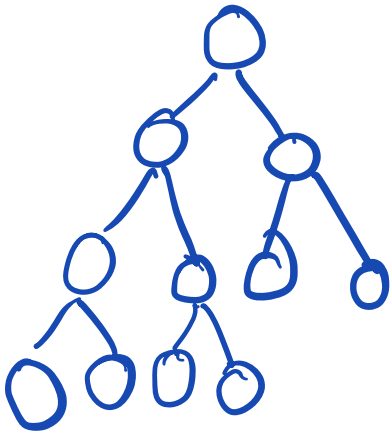
n ~~do~~ nodes

$\Theta(\log n)$

Tighter bound on Build-Max-Heap

- Previous analysis overestimated time for many of the calls to Max-Heapify
- More careful analysis:
  - Runtime of Max-Heapify is at most ch for some constant c (since it is $\theta(h)$)
  - Tree has 1 node that is root of subtree of height $\lfloor \log_2 n \rfloor$
    2 nodes that are roots of subtrees of height $\lfloor \log_2 n \rfloor - 1$
    …
    2^i nodes that are roots of subtrees of height
    …
  - $\leq 2^{\lfloor \log_2 n \rfloor}$ nodes that are roots of subtrees of height 1
  - Total runtime ~~at most~~: $\sum_{h=0}^{\lfloor \log_2 n \rfloor} 2^{\lfloor \log_2 n \rfloor - h} \cdot ch \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} 2^{\log_2 n} \cdot \frac{1}{2^h} \cdot ch = \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^h} \cdot ch \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} = 2cn$ WHY?





Max Heapify

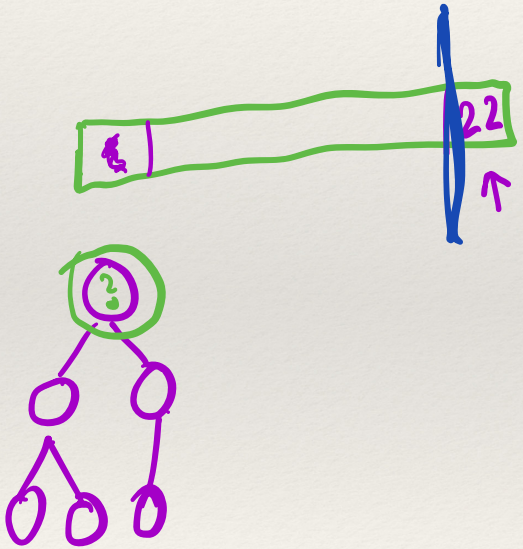
3 trees of height 1

1 tree of height 2

worst-case

- Runtime of Build-MaxHeap is $\theta(n)$

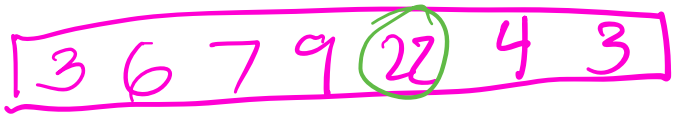
Heapsort



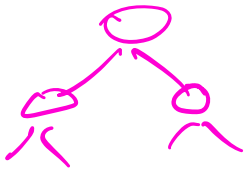

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Loop invariant:

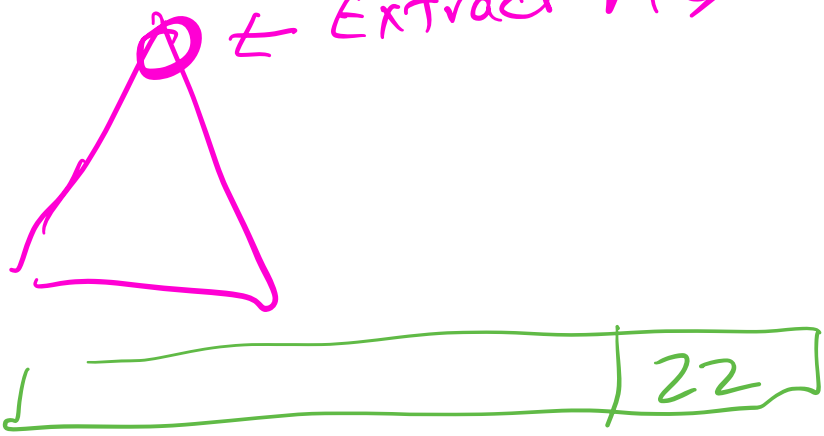
Heap only contains A[1…i]. They are the smallest items in A. Remaining elements are in A[i+1…n] in sorted order



A



Build Heap



Heapsort

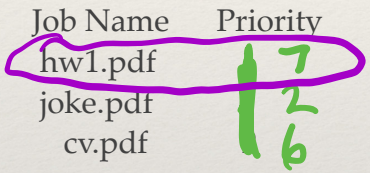
```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Runtime?

Priority Queue

Priority Queue is an Abstract Data Type

- Stores set S of elements
- Each element has a number, called a key, indicating its priority
  - e.g., Elements may be print jobs waiting to be processed



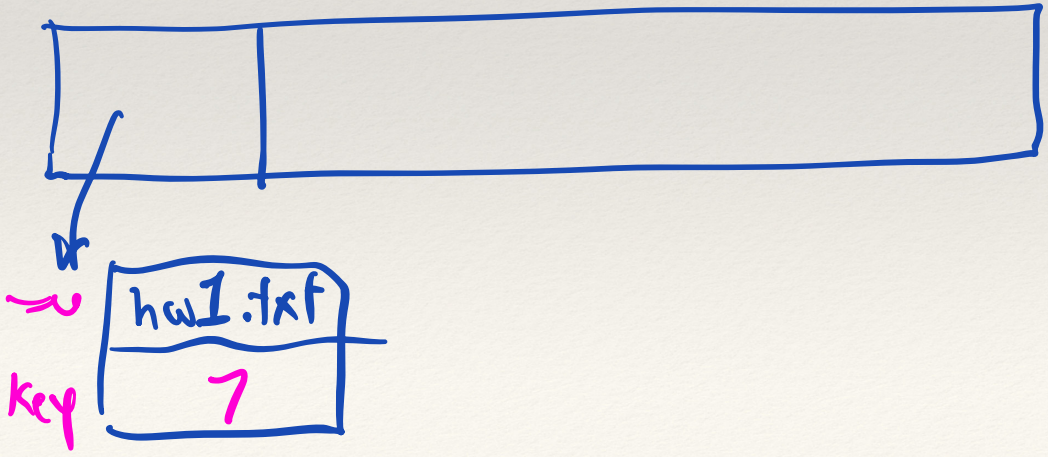
- (Max) Priority queue implementation must support the following operations
  - Insert(S,x,k) Insert element x into S with key k
  - Maximum(S) Return element in S with largest key (but don't remove)
  - Extract-Max(S) Remove and return element in S with largest key
  - Increase-Key(S,x,k) Increase key of element x of S to k (assume it is less than k)

- Could implement priority queue (PQ) using different data structures
- Example: Implementation of PQ using unsorted doubly-linked list
  - Insert(S,x,k) Insert element x into S with key k
    - How? Insert element x with key k at front of doubly-linked list
    - Runtime of Insert(S,x,k)? $\Theta(1)$ (Worst Case)
  - Maximum(S) Return element in S with largest key (but don't remove)
    - How? Linear search
    - Runtime? $\Theta(n)$ where n is size of S (Worst Case)
  - Extract-Max(S) Remove and return element in S with largest key
    - How? Linear search to find, delete from list
    - Runtime? $\Theta(n)$ for search, $\Theta(1)$ for delete (wc)
  - Increase-Key(S,x,k) Increase key of element x of S to k (assume it is less than k)
    - How? Linear search to find, increase key
    - Runtime? $\Theta(n)$ for search, $\Theta(1)$ for delete (wc)



- Usually better to implement using a **heap**
  - but need to define heap procedures to support the PQ operations
    - Insert(S,x,k)
    - Maximum(S)
    - Extract-Max(S)
    - Increase-Key(S,x,k)
  - For heap sort we only defined
    - Build-Max-Heap(A)
    - Max-Heapify(A,i)

- A[i].key is key of element in position i of heap
  - although actually array may only hold pointers to elements



Max-Heap-Maximum

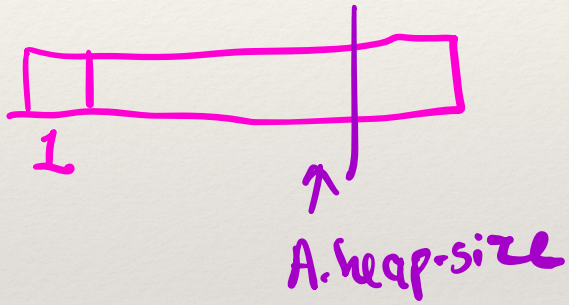
- Max-Heap-Maximum(A) // return max element, don't delete
  - if A.heapsize < 1
    - error "heap underflow"
  - return A[1]

Max-Heap-Maximum

- Max-Heap-Maximum(A) // return max element, don't delete
  - if A.heapsize < 1
    - error "heap underflow"
  - return A[1]

Max-Heap-Extract-Max(A)

- Max-Heap-Extract-Max(A)
  - max = Max-Heap-Maximum(A)
  -
  -
  -
  - return max



Max-Heap-Extract-Max(A)

- Max-Heap-Extract-Max(A)
  - max = Max-Heap-Maximum(A)
  - A[1] = A[A.heapsize] // move last element in heap to root
  - A.heapsize = A.heapsize-1 // update heapsize
  - Max-Heapify(A,1) // restore heap order
  - return max

Max-Heap-Extract-Max(A)

- Max-Heap-Extract-Max(A)
  - max = Max-Heap-Maximum(A)
  - A[1] = A[A.heapsize] // move last element in heap to root
  - A.heapsize = A.heapsize-1 // update heapsize
  - Max-Heapify(A,1) // restore heap order
  - return max

w.c. Runtime? $\Theta(\log n)$

Express runtime as function of n, where n is ~~the length of the array A~~ size of heap

IncreaseKey and "Handle"

- In order to do IncreaseKey(A,x,k), need to be able to locate element x in the array
- Implement a "handle" for x to map between elements and array positions
  - Locate element x in the heap, i.e., map elements to array indices
  - (Also, determine which element x is in a given array position, i.e.,map array positions to elements)

- Handle must be updated when elements move during operations on the heap
- One implementation of handle:
  - add array position information to the element (object) itself
- This implementation has $\theta(1)$ overhead per position change of element in heap, but storing info in element violates "abstraction barrier"

- Another implementation of handle
  - ~~add array position information to the element (object) itself~~
    Use hashtable to map elements to array positions
- Need to update information when array position of x changes. Adds overhead.
- ~~Above implementation has $\theta(1)$ overhead per position change, but storing info in element violates "abstraction barrier"~~
  Above implementation has $O(1)$ overhead "on average" per position change but worst case is $O(n)$.
  Respects abstraction barrier.

- Going forward, we'll exclude the runtime of mapping operations from the runtimes.